```
"""Thin client for the public NHL web API (api-web.nhle.com)."""
import requests

BASE = "https://api-web.nhle.com/v1"
_session = requests.Session()
_session.headers["User-Agent"] = "Faceoff/1.0 (+https://nhlanalytica.com)"

FINAL_STATES = {"FINAL", "OFF"}
LIVE_STATES = {"LIVE", "CRIT"}


def _get(path):
    r = _session.get(f"{BASE}{path}", timeout=10)
    r.raise_for_status()
    return r.json()


def _name(team):
    for key in ("commonName", "name", "placeName"):
        v = team.get(key)
        if isinstance(v, dict) and v.get("default"):
            return v["default"]
    return team.get("abbrev", "")


def _parse(g):
    home, away = g.get("homeTeam", {}), g.get("awayTeam", {})
    return {
        "id": g["id"],
        "start_utc": g.get("startTimeUTC"),
        "state": g.get("gameState", "FUT"),
        "home": home.get("abbrev"),
        "away": away.get("abbrev"),
        "home_name": _name(home),
        "away_name": _name(away),
        "home_score": home.get("score"),
        "away_score": away.get("score"),
        "period_type": (g.get("gameOutcome") or {}).get("lastPeriodType"),
    }


def games_on(date_str):
    """Regular-season and playoff games scheduled on a date (YYYY-MM-DD, ET)."""
```

```
    data = _get(f"/schedule/{date_str}")
    for day in data.get("gameWeek", []):
        if day.get("date") == date_str:
            return [_parse(g) for g in day.get("games", []) if g.get("gameType")
    return []


def standings():
    """{abbrev: (point_pct, games_played)} from current standings."""
    out = {}
    for row in _get("/standings/now").get("standings", []):
        ab = (row.get("teamAbbrev") or {}).get("default")
        if ab:
            out[ab] = (float(row.get("pointPctg") or 0.5), int(row.get("gamesPlay
    return out
```